

Package ‘LFJM’

July 31, 2026

Type Package

Title Longitudinal Functional Joint Models

Version 1.0.0

Description Functions for fitting longitudinal functional joint models when a functional predictor is observed repeatedly together with a scalar longitudinal response and a time-to-event outcome. The package uses weighted-grid longitudinal functional principal component analysis to separate persistent between-subject variation from visit-specific within-subject variation, and estimates the resulting shared-current-value joint model by adjusted profile h-likelihood. It provides tools for model fitting, control specification, prediction, fitted values, residuals, coefficient and covariance extraction, summaries, visualization, and simulation-based data generation.

Depends R (>= 3.5.0)

Imports graphics,
MASS,
stats,
survival,
tools

License GPL-3

Encoding UTF-8

ByteCompile true

Contents

coef.lfjm	2
fitted.lfjm	4
lfjm	6
lfjm_control	9
lfjm_lfpca	12
logLik.lfjm	16
plot.lfjm	17
plot.lfjm_lfpca	19
predict.lfjm	22
predict.lfjm_lfpca	25
print.lfjm	28
print.lfjm_control	29

print.lfjm_lfpca	31
print.lfjm_simulation	33
print.summary.lfjm	34
residuals.lfjm	36
simulate_lfjm	38
summary.lfjm	41
vcov.lfjm	43

Index 46

coef.lfjm	<i>Extract Coefficients from an LFJM Fit</i>
-----------	--

Description

Extracts scalar parameters, score-space coefficients, and reconstructed functional coefficient estimates from a fitted longitudinal functional joint model.

Usage

```
## S3 method for class lfjm
coef(
  object,
  type = c("all", "scalar", "between", "within", "survival", "scores"),
  ...
)
```

Arguments

object	An object of class "lfjm", typically returned by <code>lfjm</code> .
type	A character string specifying the coefficients to return. The available choices are "all", "scalar", "between", "within", "survival", and "scores". Partial matching is allowed.
...	Additional arguments. They are currently not used.

Details

The fitted model separates the centered longitudinal functional predictor into a subject-level component, $B_i(s)$, and a visit-level component, $U_{ij}(s)$. In the random-intercept implementation, the longitudinal submodel is

$$Y_{ij} = \beta_0 + \beta_{\text{time}} t_{ij} + \beta_Q Q_i + \langle B_i, \beta_B \rangle + \langle U_{ij}, \beta_U \rangle + u_i + \varepsilon_{ij}.$$

Here, β_B describes the effect of persistent between-subject functional variation, whereas β_U describes the effect of visit-specific within-subject variation.

The current value entering the survival model excludes the visit-specific functional deviation:

$$m_i(t) = \beta_0 + \beta_{\text{time}} t + \beta_Q Q_i + \langle B_i, \beta_B \rangle + u_i.$$

The Weibull proportional-hazards submodel is

$$h_i(t) = \tau t^{\tau-1} \exp\{\gamma_0 + \gamma_Q Q_i + \alpha m_i(t)\}.$$

Consequently, the functional effect induced in the survival linear predictor is $\alpha\beta_B(s)$, rather than a separate direct functional effect.

At Stage 1, longitudinal functional principal component analysis supplies between- and within-level eigenfunctions and predicted scores. At Stage 2, the adjusted profile h-likelihood estimates the retained score coefficients $\theta_{B,k}$ and $\theta_{U,l}$. The reported functional coefficients are reconstructed on the fitted functional grid as

$$\hat{\beta}_B(s) = \sum_{k=1}^{K_B} \hat{\theta}_{B,k} \hat{\phi}_k^B(s), \quad \hat{\beta}_U(s) = \sum_{l=1}^{K_U} \hat{\theta}_{U,l} \hat{\phi}_l^U(s),$$

The survival functional effect returned by `type = "survival"` is $\hat{\alpha}\hat{\beta}_B(s)$.

The possible values of `type` have the following meanings:

"scalar" Returns the finite-dimensional parameters `beta0`, `beta_time`, `beta_Q`, `sigma_e`, `sigma_u`, `gamma0`, `gamma_Q`, `alpha`, and `Tau`. The quantities `sigma_e` and `sigma_u` are standard deviations.

"between" Returns $\hat{\beta}_B(s)$ evaluated on the fitted functional grid.

"within" Returns $\hat{\beta}_U(s)$ evaluated on the fitted functional grid.

"survival" Returns $\hat{\alpha}\hat{\beta}_B(s)$ evaluated on the fitted functional grid.

"scores" Returns the retained between- and within-level score coefficients.

"all" Returns all of the preceding coefficient groups in a single list.

As in the fitted model, these estimates are conditional on the Stage-1 mean surface, eigensystems, and predicted functional scores.

Value

The returned object depends on `type`:

"scalar" A named numeric vector with elements `beta0`, `beta_time`, `beta_Q`, `sigma_e`, `sigma_u`, `gamma0`, `gamma_Q`, `alpha`, and `Tau`.

"between" A numeric vector containing the estimated between-level coefficient function on the fitted grid.

"within" A numeric vector containing the estimated within-level coefficient function on the fitted grid.

"survival" A numeric vector containing the estimated induced survival functional effect on the fitted grid.

"scores" A list with components `between` and `within`, containing the estimated score-space coefficient vectors.

"all" A list with the following components:

`scalar` The named vector of scalar parameter estimates.

`score_coefficients` A list containing the between- and within-level score coefficients.

`beta_between` The estimated between-level functional coefficient.

`beta_within` The estimated within-level functional coefficient.

`survival_between` The induced survival functional effect $\hat{\alpha}\hat{\beta}_B(s)$.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

coef(fit)
coef(fit, type = "scalar")
coef(fit, type = "between")
coef(fit, type = "within")
coef(fit, type = "survival")
coef(fit, type = "scores")

## End(Not run)
```

fitted.lfjm

Extract Longitudinal Fitted Values from an LFJM Fit

Description

Returns the subject-specific fitted values for the longitudinal outcome from a fitted longitudinal functional joint model.

Usage

```
## S3 method for class lfjm
fitted(object, ...)
```

Arguments

object	An object of class "lfjm", typically returned by lfjm .
...	Additional arguments. They are currently not used.

Details

The longitudinal functional joint model separates the centered functional predictor into a persistent subject-level component and a visit-specific component. For subject i at visit j , the fitted value returned by this method is

$$\hat{\mu}_{ij} = \hat{\beta}_0 + \hat{\beta}_{\text{time}} t_{ij} + \hat{\beta}_Q Q_i + \hat{\xi}_i^T \hat{\theta}_B + \hat{\zeta}_{ij}^T \hat{\theta}_U + \hat{u}_i.$$

Here, $\hat{\xi}_i$ and $\hat{\zeta}_{ij}$ are the between- and within-level scores obtained from Stage 1, while $\hat{\theta}_B$ and $\hat{\theta}_U$ are their Stage-2 coefficient vectors. The term $\hat{u}_i$ is the subject-specific random-intercept mode profiled in the adjusted profile h-likelihood.

These are conditional, subject-specific fitted values. They include both functional score contributions and the profiled random intercept; therefore, they differ from marginal fitted values formed from the fixed part alone. Stage-1 quantities are treated as fixed in accordance with the two-stage estimation procedure.

Value

A numeric vector containing one subject-specific fitted longitudinal value for each row of the longitudinal data stored in object. The values follow the row ordering used in the fitted model, namely subject identifier followed by visit time.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

fitted_values <- fitted(fit)
head(fitted_values)

## End(Not run)
```

lfjm

*Fit a Longitudinal Functional Joint Model***Description**

Fits a two-stage longitudinal functional joint model for a repeatedly observed functional predictor, a scalar longitudinal outcome, and a time-to-event outcome. Longitudinal functional principal component analysis separates between- and within-subject functional variation, and the joint model is estimated by adjusted profile h-likelihood.

Usage

```
lfjm(
  longitudinal_data,
  survival_data,
  functional_data,
  grid,
  id = "ID",
  visit_time = "obstime",
  response = "Y",
  longitudinal_covariate = "Q1",
  survival_time = "time",
  event = "event",
  survival_covariate = "Q1",
  control = lfjm_control()
)
```

Arguments

<code>longitudinal_data</code>	A data frame containing one row for each longitudinal visit. It must contain the subject identifier, visit time, scalar longitudinal response, and scalar longitudinal covariate specified by <code>id</code> , <code>visit_time</code> , <code>response</code> , and <code>longitudinal_covariate</code> , respectively.
<code>survival_data</code>	A data frame containing exactly one row for each subject. It must contain the subject identifier, observed event or censoring time, event indicator, and scalar survival covariate specified by <code>id</code> , <code>survival_time</code> , <code>event</code> , and <code>survival_covariate</code> , respectively.
<code>functional_data</code>	A numeric matrix containing the repeatedly observed functional predictor. Each row must correspond to the same row of <code>longitudinal_data</code> , and each column must correspond to one point in <code>grid</code> .
<code>grid</code>	A finite, strictly increasing numeric vector giving the common functional observation grid. Its length must equal the number of columns of <code>functional_data</code> .
<code>id</code>	A character string naming the subject-identifier column in both data frames.
<code>visit_time</code>	A character string naming the visit-time column in <code>longitudinal_data</code> .
<code>response</code>	A character string naming the scalar longitudinal-response column in <code>longitudinal_data</code> .
<code>longitudinal_covariate</code>	A character string naming the scalar covariate included in the longitudinal sub-model.

survival_time	A character string naming the observed event or censoring-time column in survival_data. All values must be positive and finite.
event	A character string naming the event-indicator column in survival_data. The column must contain only 0 for censoring and 1 for an observed event.
survival_covariate	A character string naming the scalar covariate entering the survival submodel directly. It may refer to the same baseline covariate used in the longitudinal submodel.
control	An object returned by <code>lfjm_control</code> , specifying LFPCA, quadrature, optimization, and standard-error settings.

Details

The method is designed for a functional predictor observed repeatedly for each subject. After removal of the estimated mean surface, the predictor is written as

$$X_{ij}(s) = \eta(s, t_{ij}) + B_i(s) + U_{ij}(s) + e_{ij}(s),$$

where $B_i(s)$ represents persistent between-subject variation and $U_{ij}(s)$ represents visit-specific within-subject variation.

Estimation proceeds in two stages. At Stage 1, `lfjm_lfpca` estimates the mean surface, the between- and within-level eigensystems, and their scores. With retained scores $\hat{\xi}_i$ and $\hat{\zeta}_{ij}$, Stage 2 fits the working longitudinal submodel

$$Y_{ij} = \beta_0 + \beta_{\text{time}} t_{ij} + \beta_Q Q_i + \hat{\xi}_i^\top \boldsymbol{\theta}_B + \hat{\zeta}_{ij}^\top \boldsymbol{\theta}_U + u_i + \varepsilon_{ij},$$

The subject-level current value is

$$m_i(t) = \beta_0 + \beta_{\text{time}} t + \beta_Q Q_i + \hat{\xi}_i^\top \boldsymbol{\theta}_B + u_i.$$

The visit-specific functional deviation is excluded from the current value because it represents transient state variation. The survival submodel is

$$h_i(t) = \tau t^{\tau-1} \exp\{\gamma_0 + \gamma_Q Q_i + \alpha m_i(t)\}, \quad \tau > 0.$$

Thus, the induced functional effect in the survival linear predictor is $\alpha \beta_B(s)$.

The Stage-2 criterion is the adjusted profile h-likelihood

$$\tilde{h}(\boldsymbol{\psi}) = \sum_i \left[h_i\{\hat{u}_i(\boldsymbol{\psi}); \boldsymbol{\psi}\} - \frac{1}{2} \log \left\{ \frac{H_i(\boldsymbol{\psi})}{2\pi} \right\} \right],$$

Here, $\hat{u}_i$ is the profiled random-intercept mode and H_i is the negative second derivative of the subject-specific h-likelihood with respect to that random intercept. The cumulative-hazard integrals are evaluated by Gauss–Legendre quadrature after a transformation that regularizes the Weibull integrand.

After estimation, the retained score coefficients are mapped back to the functional grid:

$$\hat{\beta}_B(s) = \sum_{k=1}^{K_B} \hat{\theta}_{B,k} \hat{\phi}_k^B(s), \quad \hat{\beta}_U(s) = \sum_{l=1}^{K_U} \hat{\theta}_{U,l} \hat{\phi}_l^U(s).$$

Before estimation, `longitudinal_data` and the rows of `functional_data` are jointly ordered by subject identifier and visit time. The survival data are then matched to the ordered subject identifiers. The subject identifiers appearing in the two data frames must agree exactly.

Value

An object of class "lfjm". It is a list containing the following principal components:

method A character string identifying the LFPCA–APHL estimator.

beta The estimated longitudinal coefficient vector, including the intercept, visit-time effect, scalar-covariate effect, and retained between- and within-level score coefficients.

theta_B, theta_U The retained between- and within-level score coefficient vectors.

beta_B, beta_U The reconstructed between- and within-level functional coefficients evaluated on grid.

surv_B_current The induced survival functional effect $\hat{\alpha}\hat{\beta}_B(s)$ evaluated on grid.

se_beta_B, se_beta_U, se_surv_B Pointwise conditional standard errors for the reconstructed functional coefficients and induced survival effect. These components are NULL if standard errors were not requested or could not be computed.

sigma_e, sigma_u The estimated residual and random-intercept standard deviations.

gamma0, gamma_Q, alpha, Tau The estimated survival intercept, direct scalar-covariate effect, current-value association parameter, and Weibull shape parameter.

uhat The profiled subject-specific random-intercept modes.

risk The fitted subject-specific survival linear predictors evaluated at the observed event or censoring times.

loglik, objective, npar The maximized adjusted profile h-likelihood, minimized negative criterion, and number of fitted Stage-2 parameters.

parameter_vector The complete optimization parameter vector. Positive scale parameters are represented on the log scale in this vector.

covariance_matrix The conditional Stage-2 covariance matrix obtained from the numerical Hessian. It is NULL when unavailable.

convergence, convergence_message The optimizer convergence code and optional convergence message.

lfpca The fitted Stage-1 object of class "lfjm_lfpca".

longitudinal_data, survival_data, functional_data, grid The internally ordered model data and functional grid.

call, control, variable_names The matched call, control settings, and original variable-name mapping.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
```



```

)

control <- lfjm_control(
  pve_between = 0.95,
  pve_within = 0.95,
  quadrature_points = 32,
  max_iterations = 500
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid,
  id = "ID",
  visit_time = "obstime",
  response = "Y",
  longitudinal_covariate = "Q1",
  survival_time = "time",
  event = "event",
  survival_covariate = "Q1",
  control = control
)

print(fit)
summary(fit)
coef(fit, type = "scalar")
fitted(fit)
predict(fit, type = "risk", landmark = 2)

## End(Not run)

```

lfjm_control

*Set Control Parameters for LFJM Estimation***Description**

Constructs a control object containing tuning parameters for the longitudinal functional principal component analysis and numerical settings for fitting a longitudinal functional joint model.

Usage

```

lfjm_control(
  pve_between = 0.95,
  pve_within = 0.95,
  components_between = NULL,
  components_within = NULL,
  mean_degree = 3,
  presmooth = TRUE,
  smoothing_df = 12,
  use_functional_slope = FALSE,
  quadrature_points = 32,
  max_iterations = 500,

```

```

    multistart = TRUE,
    standard_errors = TRUE
  )

```

Arguments

pve_between	A numeric value in $(0, 1]$ giving the target proportion of variance explained for selecting between-subject functional principal components. This setting is used when components_between = NULL.
pve_within	A numeric value in $(0, 1]$ giving the target proportion of variance explained for selecting within-subject functional principal components. This setting is used when components_within = NULL.
components_between	NULL, or a positive integer specifying the number of between-subject functional principal components to retain. A supplied value overrides selection based on pve_between.
components_within	NULL, or a positive integer specifying the number of within-subject functional principal components to retain. A supplied value overrides selection based on pve_within.
mean_degree	A non-negative number controlling the polynomial degree used to estimate the functional mean surface over visit time. The effective degree is also restricted by the number of distinct visit times and has a maximum of 3.
presmooth	A logical value indicating whether each observed functional curve should be smoothed before the longitudinal functional principal component analysis.
smoothing_df	The degrees of freedom supplied to smooth.spline when presmooth = TRUE.
use_functional_slope	A logical value indicating whether the between-subject decomposition should use subject-specific functional intercepts and functional slopes. If FALSE, only subject-specific functional intercepts are used.
quadrature_points	A numeric value of at least 2 giving the number of Gauss–Legendre quadrature points used to approximate the cumulative hazard contribution. The value is stored as an integer.
max_iterations	A positive numeric value giving the maximum number of iterations allowed for each bounded optimization run. The value is stored as an integer.
multistart	A logical value indicating whether optimization should be performed from multiple initial values for the association parameter. The solution with the best objective value is retained and further refined.
standard_errors	A logical value indicating whether conditional standard errors should be calculated from a numerical Hessian after model fitting.

Details

The longitudinal functional joint model is estimated in two stages. In the first stage, longitudinal functional principal component analysis separates the functional variation into between-subject and within-subject components. If a component count is not fixed explicitly, the smallest number of components whose cumulative eigenvalue proportion reaches the corresponding pve_between or pve_within threshold is retained.

With `use_functional_slope = TRUE`, the between-subject process is represented using a subject-specific functional intercept and slope,

$$B_i(s; t) = B_i^{(0)}(s) + (t - \bar{t})B_i^{(1)}(s),$$

where s indexes the functional domain and t is visit time. With the default setting, the between-subject decomposition is based on a subject-specific functional intercept only.

In the second stage, the joint model is fitted using an adjusted profile h-likelihood. Gauss–Legendre quadrature approximates the time integral in the survival contribution. Increasing `quadrature_points` may improve the numerical approximation at the cost of additional computation. `multistart` and `max_iterations` affect numerical optimization, not the statistical specification of the model.

When requested, standard errors are based on the numerical Hessian from the second-stage fit. They are conditional on the estimated functional quantities from the first stage and therefore do not propagate first-stage estimation uncertainty.

Logical controls are stored using `isTRUE`; consequently, only an exact `TRUE` value activates the corresponding option.

Value

An object of class "lfjm_control", represented by a list with the following components:

`pve_between` Target proportion of between-subject variance explained.
`pve_within` Target proportion of within-subject variance explained.
`components_between` Fixed number of between-subject components, or `NULL`.
`components_within` Fixed number of within-subject components, or `NULL`.
`mean_degree` Control for the polynomial degree of the functional mean surface.
`presmooth` Whether functional observations are presmoothed.
`smoothing_df` Degrees of freedom used for presmoothing.
`use_functional_slope` Whether functional slopes are included in the between-subject decomposition.
`quadrature_points` Integer number of quadrature points.
`max_iterations` Integer maximum number of optimization iterations.
`multistart` Whether multiple optimization starts are used.
`standard_errors` Whether conditional standard errors are requested.

Examples

```
## Not run:
default_control <- lfjm_control()
default_control

custom_control <- lfjm_control(
  pve_between = 0.99,
  pve_within = 0.95,
  components_between = 3,
  components_within = 2,
  mean_degree = 3,
  presmooth = TRUE,
  smoothing_df = 12,
  use_functional_slope = FALSE,
```

```

    quadrature_points = 48,
    max_iterations = 1000,
    multistart = TRUE,
    standard_errors = TRUE
)

custom_control$components_between
custom_control$quadrature_points

## End(Not run)

```

lfjm_lfpca

Longitudinal Functional Principal Component Analysis

Description

Performs the first-stage longitudinal functional principal component analysis used by the longitudinal functional joint model. The procedure estimates a time-varying functional mean, separates between-subject and within-subject variation, and obtains weighted-grid eigenfunctions, scores, and reconstructed functional components.

Usage

```

lfjm_lfpca(
  functional_data,
  subject,
  time,
  grid,
  pve_between = 0.95,
  pve_within = 0.95,
  components_between = NULL,
  components_within = NULL,
  mean_degree = 4,
  presmooth = TRUE,
  smoothing_df = 12,
  use_functional_slope = FALSE
)

```

Arguments

functional_data	A numeric matrix containing the repeatedly observed functional predictor. Rows correspond to subject visits and columns correspond to the locations in grid.
subject	A vector of subject identifiers with one element for each row of functional_data. Repeated identifiers indicate multiple visits from the same subject.
time	A numeric vector of visit times with one element for each row of functional_data.
grid	A finite, strictly increasing numeric vector giving the common functional grid. Its length must equal the number of columns of functional_data.
pve_between	A numeric value in (0, 1] giving the target proportion of between-subject variance explained. It is used when components_between = NULL.

pve_within	A numeric value in $(0, 1]$ giving the target proportion of within-subject variance explained. It is used when components_within = NULL.
components_between	NULL, or a positive integer specifying the number of between-subject components to retain. A supplied value overrides pve_between.
components_within	NULL, or a positive integer specifying the number of within-subject components to retain. A supplied value overrides pve_within.
mean_degree	A non-negative number controlling the polynomial degree used for the time-varying functional mean. The effective degree cannot exceed 3 or the number of distinct visit times minus one.
presmooth	A logical value indicating whether each observed curve is smoothed before the decomposition.
smoothing_df	The degrees of freedom supplied to <code>smooth.spline</code> when presmooth = TRUE.
use_functional_slope	A logical value indicating whether the between-subject process should contain subject-specific functional intercepts and functional slopes. If FALSE, the between-subject process is represented by a functional intercept only.

Details

The working two-level representation of the repeatedly observed functional predictor is

$$X_{ij}(s) = \eta(s, t_{ij}) + B_i(s) + U_{ij}(s),$$

where $\eta(s, t)$ is the population mean surface, $B_i(s)$ is the stable subject-level component, and $U_{ij}(s)$ is the visit-specific component. This separation permits the later joint model to assign distinct functional effects to persistent between-subject variation and changing within-subject variation.

The function first standardizes visit time as

$$z_{ij} = \{t_{ij} - \bar{t}\} / s_t.$$

If the sample standard deviation of visit time is zero or numerically negligible, it is replaced by one. At every grid location, the mean surface is estimated by polynomial regression on standardized time. Its effective degree is the minimum of 3, mean_degree, and the number of distinct visit times minus one. The fitted mean surface is then removed from every curve.

When use_functional_slope = FALSE, the subject-level curve is estimated by the average centered curve for that subject. The within-subject curves are the visit-level residuals obtained after removing this subject average. Thus, on the observed grid,

$$\tilde{X}_{ij}(s) = \hat{B}_i^{(0)}(s) + \hat{U}_{ij}(s).$$

When use_functional_slope = TRUE, a functional intercept and slope are estimated separately for each subject by regressing its centered curves on $(1, z_{ij})$ at all grid locations. The decomposition becomes

$$\tilde{X}_{ij}(s) = \hat{B}_i^{(0)}(s) + z_{ij} \hat{B}_i^{(1)}(s) + \hat{U}_{ij}(s).$$

For a subject with only one visit, the observed centered curve is used as the functional intercept and the functional slope is set to zero.

Separate weighted principal component analyses are applied to the between-subject and within-subject quantities. If $\mathbf{W}$ is the diagonal matrix of trapezoidal integration weights, the eigenfunctions are normalized in the discrete functional inner product

$$\langle f, g \rangle_{\mathbf{W}} = f^T \mathbf{W} g.$$

Consequently, the returned eigenfunctions are orthonormal with respect to the quadrature weights rather than the unweighted Euclidean inner product. With functional slopes, the between-subject analysis is performed on the concatenated intercept and slope curves in the direct-sum space, using the trapezoidal weights for both parts. Each between eigencomponent therefore consists of the paired functions returned in `phi0` and `phi1`.

Unless the number of components is supplied explicitly, the smallest number whose cumulative eigenvalue proportion reaches the specified proportion of variance explained is retained. Eigenvalues smaller than 10^{-10} times the largest eigenvalue are treated as numerically zero. The returned visit-level quantities remain in the original input row order, whereas subject-level quantities follow the sorted unique subject identifiers in `ids`.

Value

An object of class "lfjm_lfpca", represented by a list with the following components:

`call` The matched function call.

`ids` The sorted unique subject identifiers.

`subject_index` Integer indices mapping the input rows to the entries of `ids`.

`grid` The functional grid.

`sgrid` A duplicate of `grid` retained for subsequent model-fitting routines.

`weights` Trapezoidal integration weights on the functional grid.

`phi0` A grid-by-KB matrix containing the functional-intercept part of the between-subject eigenfunctions.

`phi1` A grid-by-KB matrix containing the functional-slope part of the between-subject eigenfunctions. It is zero when `use_functional_slope = FALSE`.

`phiU` A grid-by-KU matrix of within-subject eigenfunctions.

`scoreB_subject` A subject-by-KB matrix of between-subject scores. Its row names are the subject identifiers.

`scoreU_visit` A visit-by-KU matrix of within-subject scores in input row order.

`B0_hat_subject` A subject-by-grid matrix of reconstructed subject-specific functional intercepts.

`B_hat_visit` A visit-by-grid matrix of reconstructed between-subject curves evaluated at each visit time.

`U_hat_visit` A visit-by-grid matrix of reconstructed within-subject curves.

`eigB` The retained between-subject eigenvalues.

`eigU` The retained within-subject eigenvalues.

`KB` The number of retained between-subject components.

`KU` The number of retained within-subject components.

`time_mean` The sample mean of the visit times used for standardization.

`time_sd` The sample standard deviation used for time standardization, or one when the original standard deviation is unavailable or numerically negligible.

`eta_coef` The polynomial coefficient matrix for the estimated functional mean surface.

eta_degree The effective polynomial degree used for the mean surface.

center_B0 The gridwise centering curve used in the between functional-intercept analysis.

center_B1 The gridwise centering curve used in the between functional-slope analysis. It is zero when functional slopes are not used.

center_U The gridwise centering curve used in the within-subject analysis.

presmooth The supplied curve-presmoothing indicator.

smooth_df The supplied smoothing degrees of freedom.

use_slope The supplied functional-slope indicator.

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 31)
subject <- rep(1:6, each = 3)
time <- rep(c(0, 1, 2), times = 6)

subject_effect <- seq(-0.75, 0.75, length.out = 6)[subject]

functional_data <-
  matrix(
    0.25 * sin(pi * grid),
    nrow = length(subject),
    ncol = length(grid),
    byrow = TRUE
  ) +
  outer(subject_effect, sin(pi * grid)) +
  outer(0.20 * time, cos(2 * pi * grid)) +
  matrix(
    rnorm(length(subject) * length(grid), sd = 0.05),
    nrow = length(subject)
  )

lfpca_fit <- lfjm_lfpca(
  functional_data = functional_data,
  subject = subject,
  time = time,
  grid = grid,
  components_between = 2,
  components_within = 2,
  mean_degree = 3,
  presmooth = FALSE
)

print(lfpca_fit)
c(between = lfpca_fit$KB, within = lfpca_fit$KU)
dim(lfpca_fit$scoreB_subject)
dim(lfpca_fit$scoreU_visit)

## End(Not run)
```

logLik.lfjm

*Extract the Adjusted Log-Likelihood from an LFJM Fit***Description**

Extracts the maximized adjusted profile h-likelihood criterion from a fitted longitudinal functional joint model and returns it as an object of class "logLik".

Usage

```
## S3 method for class lfjm
logLik(object, ...)
```

Arguments

object An object of class "lfjm", typically returned by `lfjm`.
 ... Additional arguments. They are currently not used.

Details

The longitudinal functional joint model contains a subject-specific random intercept that is shared by the longitudinal and survival submodels. Let $h_i(u_i; \psi)$ denote the hierarchical log-likelihood contribution of subject i , including its longitudinal, survival, and random-effect terms. For fixed model parameters ψ , the random-intercept mode is

$$\hat{u}_i = \arg \max_{u_i} h_i(u_i; \psi).$$

The fitted criterion is the adjusted profile h-likelihood

$$\tilde{h}(\psi) = \sum_{i=1}^N \left[h_i(\hat{u}_i; \psi) - \frac{1}{2} \log \left\{ \frac{H_i}{2\pi} \right\} \right],$$

Here, $H_i = -\partial^2 h_i(u_i; \psi) / \partial u_i^2$ evaluated at $\hat{u}_i$ is the positive curvature of the hierarchical log-likelihood. In the implemented random-intercept model,

$$H_i = \frac{n_i}{\sigma_e^2} + \alpha^2 \Lambda_i + \frac{1}{\sigma_u^2},$$

where n_i is the number of longitudinal observations and Λ_i is the fitted cumulative hazard contribution for subject i .

This adjustment is the first-order Laplace approximation to the marginal log-likelihood obtained by integrating out the random intercept:

$$\log \int \exp\{h_i(u_i; \psi)\} du_i = h_i(\hat{u}_i; \psi) - \frac{1}{2} \log \left\{ \frac{H_i}{2\pi} \right\} + R_i.$$

The returned criterion omits the higher-order remainder R_i . When the association parameter $\alpha = 0$, the hierarchical log-likelihood is quadratic in the random intercept, $R_i = 0$, and the adjustment is exact. For nonzero association, the returned value is an adjusted profile h-likelihood approximation rather than an exactly marginalized log-likelihood.

The "df" attribute equals `object$np`, the length of the fitted Stage-2 parameter vector. It includes the longitudinal score coefficients and finite-dimensional model parameters but excludes the profiled subject-specific random intercepts and all Stage-1 LFPCA quantities. The "nobs" attribute equals the number of rows in `object$longitudinal_data`; it therefore counts longitudinal visit-level observations rather than independent subjects or survival records.

Value

A numeric value of class "logLik" containing the maximized adjusted profile h-likelihood stored in object\$loglik. It has the following attributes:

df The number of fitted Stage-2 parameters, obtained from object\$npar.

nobs The number of rows in the longitudinal data stored in the fitted object.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

adjusted_loglikelihood <- logLik(fit)
adjusted_loglikelihood
as.numeric(adjusted_loglikelihood)
attr(adjusted_loglikelihood, "df")
attr(adjusted_loglikelihood, "nobs")

AIC(fit)
BIC(fit)

## End(Not run)
```

Description

Produces plots of the estimated functional coefficient functions or the between- and within-level eigensystems from a fitted longitudinal functional joint model.

Usage

```
## S3 method for class lfjm
plot(
  x,
  type = c("coefficients", "eigenfunctions", "eigenvalues"),
  level = c("between", "within"),
  ...
)
```

Arguments

x	An object of class "lfjm", typically returned by <code>lfjm</code> .
type	A character string specifying the plot to produce. The available choices are "coefficients", "eigenfunctions", and "eigenvalues". Partial matching is allowed.
level	A character string specifying whether the between-subject or within-subject eigensystem is plotted. The available choices are "between" and "within". This argument is validated but otherwise ignored when type = "coefficients".
...	Additional graphical arguments passed to <code>matplot</code> for coefficient and eigenfunction plots, or to <code>plot</code> for eigenvalue plots. Arguments that duplicate graphical parameters fixed internally by the method cannot be used to override those parameters.

Details

The fitted model separates the centered longitudinal functional predictor into a persistent between-subject process $B_i(s)$ and a visit-specific within-subject process $U_{ij}(s)$. Their effects in the longitudinal submodel are represented by two coefficient functions, $\beta_B(s)$ and $\beta_U(s)$. The shared current-value association routes the between-subject functional effect to the survival submodel through $\alpha\beta_B(s)$.

With type = "coefficients", the method plots the three estimated functions stored in the fitted object:

$$\hat{\beta}_B(s), \quad \hat{\beta}_U(s), \quad \hat{\alpha}\hat{\beta}_B(s).$$

They are evaluated at `x$grid`. The between-subject longitudinal coefficient is shown by a solid black line, the within-subject longitudinal coefficient by a dashed blue line, and the induced survival coefficient by a dotted red line.

The functional coefficients are obtained by mapping the estimated Stage-2 score coefficients back to the functional grid. For the retained centered representations $\hat{B}_{i,K_B} = \sum_k \hat{\xi}_{ik} \hat{\phi}_k^B$ and $\hat{U}_{ij,K_U} = \sum_l \hat{\zeta}_{ijl} \hat{\phi}_l^U$, the trapezoidal-grid inner product gives

$$\langle \hat{B}_{i,K_B}, \hat{\beta}_B \rangle_{\mathbf{W}} = \hat{\xi}_i^T \hat{\theta}_B, \quad \langle \hat{U}_{ij,K_U}, \hat{\beta}_U \rangle_{\mathbf{W}} = \hat{\zeta}_{ij}^T \hat{\theta}_U.$$

Thus, the coefficient plot displays the estimated functions themselves; a subject- or visit-specific functional contribution additionally depends on the corresponding functional curve or scores.

With type = "eigenfunctions" or type = "eigenvalues", plotting is delegated to the Stage-1 object stored in `x$lfpca`. For level = "between", the eigenfunction plot displays the columns of `x$lfpca$phi0`, namely the functional-intercept part of the retained between-subject eigenfunctions. If functional slopes were used in Stage 1, their phi1 parts are not displayed by this method. For level = "within", the columns of `x$lfpca$phiU` are displayed.

The eigenvalue plot shows the retained eigenvalues in component order, using `x$lfpca$eigB` or `x$lfpca$eigU`. Eigenvalues quantify the variation associated with the retained directions at the selected level. If no components were retained at that level, an empty plotting region is produced with an explanatory title.

Value

The original fitted object `x`, returned invisibly. The function is called for its graphical side effect.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

plot(fit)
plot(fit, type = "coefficients", ylim = c(-2, 2))
plot(fit, type = "eigenfunctions", level = "between")
plot(fit, type = "eigenfunctions", level = "within")
plot(fit, type = "eigenvalues", level = "between")
plot(fit, type = "eigenvalues", level = "within")

## End(Not run)
```

plot.lfjm_lfpca

Plot Longitudinal Functional PCA Eigensystems

Description

Plots the retained eigenfunctions or eigenvalues from the between-subject or within-subject level of a longitudinal functional principal component analysis.

Usage

```
## S3 method for class lfjm_lfpca
plot(
  x,
  type = c("eigenfunctions", "eigenvalues"),
  level = c("between", "within"),
  ...
)
```

Arguments

<code>x</code>	An object of class "lfjm_lfpca", typically returned by <code>lfjm_lfpca</code> .
<code>type</code>	A character string specifying whether to plot the retained "eigenfunctions" or "eigenvalues". Partial matching is allowed.
<code>level</code>	A character string specifying the decomposition level. Use "between" for the subject-level eigensystem or "within" for the visit-level eigensystem. Partial matching is allowed.
<code>...</code>	Additional graphical arguments passed to <code>matplot</code> for eigenfunction plots or to <code>plot</code> for eigenvalue plots. Arguments that duplicate graphical parameters fixed internally by the method cannot be used to override those parameters.

Details

The longitudinal functional principal component analysis separates the centered functional predictor into between-subject and within-subject processes. In the functional-intercept formulation,

$$\tilde{X}_{ij}(s) = B_i(s) + U_{ij}(s),$$

where $B_i(s)$ describes persistent variation among subjects and $U_{ij}(s)$ describes variation of visit j around the corresponding subject-level curve.

The two processes are represented by separate truncated Karhunen–Loeve expansions,

$$B_i(s) \simeq \sum_{k=1}^{K_B} \xi_{ik} \phi_k^B(s), \quad U_{ij}(s) \simeq \sum_{l=1}^{K_U} \zeta_{ijl} \phi_l^U(s).$$

The between-subject eigenfunctions are stored in `x$phi0`, the within-subject eigenfunctions in `x$phiU`, and their retained eigenvalues in `x$eigB` and `x$eigU`, respectively.

With `type = "eigenfunctions"`, all retained eigenfunctions at the selected level are drawn as lines over `x$grid`. The eigenfunctions were obtained from a weighted-grid decomposition using the trapezoidal integration weights w_d . Their discrete orthonormality is therefore

$$\sum_d w_d \phi_k(s_d) \phi_{k'}(s_d) = \delta_{kk'},$$

They need not be orthonormal under the ordinary unweighted Euclidean inner product.

For `level = "between"`, the method plots `x$phi0`. When the LFPCA was fitted with `use_functional_slope = TRUE`, each between-subject component consists of a functional-intercept part in `x$phi0` and a functional-slope part in `x$phi1`; this plotting method displays only the functional-intercept part. For `level = "within"`, it plots `x$phiU`. In the functional-slope case, the between-level eigenvalues refer to the complete intercept–slope components in the direct-sum space, even though only their intercept parts are drawn in the eigenfunction plot.

With `type = "eigenvalues"`, the method plots the retained eigenvalues against their component indices. Eigenvalues quantify the level-specific variation associated with each retained direction and appear in decreasing order. The plot contains only the components retained by the fixed-component or proportion-of-variance rule used in `lfjm_lfpca`; it does not display discarded eigenvalues.

If no components are available at the requested level, the method creates an empty plotting region with a title stating that no components were retained.

Value

The original `"lfjm_lfpca"` object `x`, returned invisibly. The function is called for its graphical side effect.

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 31)
subject <- rep(1:6, each = 3)
time <- rep(c(0, 1, 2), times = 6)

subject_effect <- seq(-0.75, 0.75, length.out = 6)[subject]

functional_data <-
  matrix(
    0.25 * sin(pi * grid),
    nrow = length(subject),
    ncol = length(grid),
    byrow = TRUE
  ) +
  outer(subject_effect, sin(pi * grid)) +
  outer(0.20 * time, cos(2 * pi * grid)) +
  matrix(
    rnorm(length(subject) * length(grid), sd = 0.05),
    nrow = length(subject)
  )

lfpca_fit <- lfjm_lfpca(
  functional_data = functional_data,
  subject = subject,
  time = time,
  grid = grid,
  components_between = 2,
  components_within = 2,
  mean_degree = 3,
  presmooth = FALSE
)

plot(lfpca_fit, type = "eigenfunctions", level = "between")
plot(lfpca_fit, type = "eigenfunctions", level = "within")
plot(lfpca_fit, type = "eigenvalues", level = "between")
plot(lfpca_fit, type = "eigenvalues", level = "within")

## End(Not run)
```

predict.lfjm

*Obtain Predictions from an LFJM Fit***Description**

Extracts in-sample longitudinal fitted values, fitted functional scores, or subject-specific random intercepts, and computes landmark-specific current values and survival risk scores from a fitted longitudinal functional joint model.

Usage

```
## S3 method for class lfjm
predict(
  object,
  type = c(
    "longitudinal",
    "current_value",
    "risk",
    "random_effects",
    "scores"
  ),
  landmark = NULL,
  ...
)
```

Arguments

object	An object of class "lfjm", typically returned by <code>lfjm</code> .
type	A character string specifying the requested prediction. The available choices are "longitudinal", "current_value", "risk", "random_effects", and "scores". Partial matching is allowed.
landmark	NULL, or a finite numeric scalar giving the landmark time. A landmark is required for type = "current_value" and type = "risk". It is optional for type = "random_effects" and determines whether full-fit or history-based random intercepts are returned. It is ignored for type = "longitudinal" and type = "scores".
...	Additional arguments. They are currently not used.

Details

The longitudinal functional joint model uses the conditional longitudinal mean

$$\mu_{ij} = \beta_0 + \beta_{\text{time}} t_{ij} + \beta_Q Q_{ij} + \boldsymbol{\xi}_i^T \boldsymbol{\theta}_B + \boldsymbol{\zeta}_{ij}^T \boldsymbol{\theta}_U + u_i,$$

Here, $\boldsymbol{\xi}_i$ and $\boldsymbol{\zeta}_{ij}$ are the between- and within-level scores from Stage 1, while u_i is the shared subject-specific random intercept profiled during Stage-2 adjusted profile h-likelihood estimation.

type = "longitudinal" returns the conditional fitted values $\hat{\mu}_{ij}$ for the stored longitudinal observations. These values use the original fitted scores and the full-fit random-intercept modes in `object$uhat`. They include both between- and within-level functional contributions.

type = "scores" returns the fitted Stage-1 score matrices without modification. The between-level matrix has one row per subject, whereas the within-level matrix has one row per longitudinal visit. When type = "random_effects" and landmark = NULL, the method returns object\$uhat. These are the random-intercept modes obtained from the full joint-model fit and therefore use all information entering that fit.

Landmark predictions: For landmark time L , the method retains stored longitudinal and functional observations satisfying $t_{ij} \leq L$. It then projects these truncated functional histories onto the fitted Stage-1 mean surface and eigensystems. Thus, the between-subject scores used for landmark prediction are re-estimated from the functional history available by L , rather than copied from the full-history fit.

Let $n_i(L)$ be the number of longitudinal measurements available for subject i by L , and let $\hat{\mu}_{ij,L}^{\text{fix}}$ denote the longitudinal predictor formed from the scalar terms and the reprojected between- and within-level scores, excluding the random intercept. The history-based random-intercept predictor is

$$\hat{u}_i(L) = \frac{\sum_{j:t_{ij} \leq L} \{Y_{ij} - \hat{\mu}_{ij,L}^{\text{fix}}\}}{n_i(L) + \hat{\sigma}_e^2 / \hat{\sigma}_u^2}.$$

This is the Gaussian random-intercept shrinkage predictor based on the longitudinal history available by the landmark. It is returned when type = "random_effects" and a landmark is supplied.

The shared current value at the landmark is

$$\hat{m}_i(L) = \hat{\beta}_0 + \hat{\beta}_{\text{time}}L + \hat{\beta}_Q Q_i + \hat{\xi}_i(L)^\top \hat{\theta}_B + \hat{u}_i(L).$$

Consistent with the joint-model specification, the transient within-visit term does not enter the smooth current value used by the survival submodel. This quantity is returned by type = "current_value".

For type = "risk", the returned value is the survival linear predictor

$$\hat{r}_i(L) = \hat{\gamma}_0 + \hat{\gamma}_Q Q_i + \hat{\alpha} \hat{m}_i(L).$$

It is a relative log-risk score. It does not include the Weibull baseline log-hazard and is not a hazard, event probability, or survival probability.

Landmark results are produced for the subject identifiers stored in the survival data. A subject with no longitudinal observation by L receives NA. An error is raised if the requested landmark precedes all stored longitudinal observations, so that no history is available for any subject. The function does not restrict the returned subjects to those still at risk at L .

Value

The returned object depends on type:

"longitudinal" An unnamed numeric vector with one conditional fitted value for each row of object\$longitudinal_data.

"current_value" A named numeric vector of landmark-specific current values, with names given by the subject identifiers in object\$survival_data.

"risk" A named numeric vector of landmark-specific relative log-risk scores, with names given by the subject identifiers in object\$survival_data.

"random_effects" A named numeric vector of subject-specific random intercepts. With landmark = NULL, these are the full-fit APHL modes. With a supplied landmark, they are the history-based shrinkage predictors $\hat{u}_i(L)$.

"scores" A list with components between and within. These contain, respectively, object\$lfpcascoreB_subject and object\$lfpcascoreU_visit.

Examples

```

## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

longitudinal_predictions <- predict(
  fit,
  type = "longitudinal"
)

full_random_intercepts <- predict(
  fit,
  type = "random_effects"
)

current_values <- predict(
  fit,
  type = "current_value",
  landmark = 2
)

risk_scores <- predict(
  fit,
  type = "risk",
  landmark = 2
)

landmark_random_intercepts <- predict(
  fit,
  type = "random_effects",
  landmark = 2
)

functional_scores <- predict(
  fit,
  type = "scores"
)

```



```

head(longitudinal_predictions)
head(current_values)
head(risk_scores)
head(landmark_random_intercepts)
dim(functional_scores$between)
dim(functional_scores$within)

## End(Not run)

```

predict.lfjm_lfpca *Project Functional Histories onto an LFPCA Fit*

Description

Projects new repeatedly observed functional histories onto the mean surface and between- and within-level eigensystems of a fitted longitudinal functional principal component analysis.

Usage

```

## S3 method for class lfjm_lfpca
predict(
  object,
  functional_data,
  subject,
  time,
  ...
)

```

Arguments

object	An object of class "lfjm_lfpca", typically returned by lfjm_lfpca .
functional_data	A numeric matrix containing the new functional observations. Rows correspond to subject visits and columns must correspond, in the same order, to the fitted grid stored in object\$grid.
subject	A vector of subject identifiers with one element for each row of functional_data. Repeated identifiers define the functional history supplied for each subject.
time	A numeric vector of visit times with one element for each row of functional_data.
...	Additional arguments. They are currently not used.

Details

This method computes functional principal component scores using an existing LFPCA fit. It does not re-estimate the functional mean surface, centering curves, eigenfunctions, eigenvalues, or truncation dimensions.

The new functional observations must be evaluated at the same grid locations and in the same column order as the training curves. The method checks the number of grid columns but, because a separate new grid is not supplied, it cannot verify the locations themselves. If the original analysis used curve presmoothing, each new curve is smoothed using the fitted grid and the stored degrees of freedom before projection.

Visit times are standardized using the training-sample quantities,

$$z_{ij}^{\text{new}} = \frac{t_{ij}^{\text{new}} - \bar{t}_{\text{fit}}}{s_{t,\text{fit}}}.$$

The fitted polynomial mean surface is evaluated at these standardized times and removed from the new curves:

$$\tilde{X}_{ij}^{\text{new}}(s) = X_{ij}^{\text{new}}(s) - \hat{\eta}(s, t_{ij}^{\text{new}}).$$

Functional-intercept projection: When the fitted object has `use_slope = FALSE`, the subject-level component is the average of the centered curves supplied for that subject,

$$\hat{B}_i^{(0)}(s) = \frac{1}{n_i} \sum_{j=1}^{n_i} \tilde{X}_{ij}^{\text{new}}(s),$$

and the visit-level component is

$$\hat{U}_{ij}(s) = \tilde{X}_{ij}^{\text{new}}(s) - \hat{B}_i^{(0)}(s).$$

Let $\mathbf{W}$ be the diagonal matrix of trapezoidal weights, let Φ_0 contain the fitted between-level eigenfunctions, and let Φ_U contain the fitted within-level eigenfunctions. The projected scores are

$$\hat{\xi}_i = \{\hat{\mathbf{B}}_i^{(0)} - \mathbf{c}_{B0}\}^\top \mathbf{W} \Phi_0, \quad \hat{\zeta}_{ij} = \{\hat{\mathbf{U}}_{ij} - \mathbf{c}_U\}^\top \mathbf{W} \Phi_U,$$

The centering curves $\mathbf{c}_{B0}$ and $\mathbf{c}_U$ are those estimated in the original fit.

Functional-intercept and slope projection: When the fitted object has `use_slope = TRUE`, the centered curves of each new subject are regressed pointwise on $(1, z_{ij}^{\text{new}})$. This produces a subject-specific functional intercept $\hat{B}_i^{(0)}$, a functional slope $\hat{B}_i^{(1)}$, and within-subject residual curves. For a subject with one supplied visit, the centered curve is used as the functional intercept and the functional slope is set to zero.

The between-level score projection is performed in the direct-sum intercept–slope space:

$$\hat{\xi}_i = \begin{bmatrix} \hat{\mathbf{B}}_i^{(0)} - \mathbf{c}_{B0} \\ \hat{\mathbf{B}}_i^{(1)} - \mathbf{c}_{B1} \end{bmatrix}^\top (\mathbf{W} \oplus \mathbf{W}) \begin{bmatrix} \Phi_0 \\ \Phi_1 \end{bmatrix}.$$

The within-level scores use the same weighted projection as in the functional-intercept case, applied to the residual curves from the subject-specific intercept–slope regression.

The between-level scores depend on the complete set of visits supplied for a new subject. Consequently, adding another visit for that subject can change both its estimated between-level score and the within-level scores of its earlier visits. Subject-level output follows the sorted unique identifiers, whereas visit-level output remains in the input row order.

Value

A list with the following components:

`ids` The sorted unique subject identifiers in the supplied data.

`subject_index` Integer indices mapping each input row to the corresponding entry in `ids`.

`scoreB_subject` A subject-by-object\$KB matrix of projected between-level scores. Row names are the subject identifiers.

`scoreU_visit` A visit-by-object\$KU matrix of projected within-level scores in the original input row order.

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 31)
subject <- rep(1:6, each = 3)
time <- rep(c(0, 1, 2), times = 6)

subject_effect <- seq(-0.75, 0.75, length.out = 6)[subject]

functional_data <-
  matrix(
    0.25 * sin(pi * grid),
    nrow = length(subject),
    ncol = length(grid),
    byrow = TRUE
  ) +
  outer(subject_effect, sin(pi * grid)) +
  outer(0.20 * time, cos(2 * pi * grid)) +
  matrix(
    rnorm(length(subject) * length(grid), sd = 0.05),
    nrow = length(subject)
  )

lfpca_fit <- lfjm_lfpca(
  functional_data = functional_data,
  subject = subject,
  time = time,
  grid = grid,
  components_between = 2,
  components_within = 2,
  mean_degree = 3,
  presmooth = FALSE
)

new_subject <- rep(c("A", "B"), each = 3)
new_time <- rep(c(0, 1, 2), times = 2)
new_subject_effect <- rep(c(-0.30, 0.50), each = 3)

new_functional_data <-
  matrix(
    0.25 * sin(pi * grid),
    nrow = length(new_subject),
    ncol = length(grid),
    byrow = TRUE
  ) +
  outer(new_subject_effect, sin(pi * grid)) +
  outer(0.20 * new_time, cos(2 * pi * grid)) +
  matrix(
    rnorm(length(new_subject) * length(grid), sd = 0.05),
    nrow = length(new_subject)
  )

projected_scores <- predict(
  lfpca_fit,
  functional_data = new_functional_data,
  subject = new_subject,
  time = new_time
)
```

```
)

projected_scores$ids
projected_scores$subject_index
projected_scores$scoreB_subject
projected_scores$scoreU_visit

## End(Not run)
```

print.lfjm

Print a Longitudinal Functional Joint Model Fit

Description

Prints a compact overview of a fitted longitudinal functional joint model, including sample sizes, retained functional component counts, the adjusted profile h-likelihood value, and the optimizer convergence code.

Usage

```
## S3 method for class lfjm
print(x, ...)
```

Arguments

x An object of class "lfjm", typically returned by [lfjm](#).

... Additional arguments. They are currently not used.

Details

The method identifies the fitted procedure as LFPCA–APHL, reflecting its two-stage construction. In Stage 1, longitudinal functional principal component analysis separates the repeatedly observed functional predictor into between-subject and within-subject components. The displayed component counts are `x$lfpca$KB` and `x$lfpca$KU`, the numbers of retained directions at these two levels.

Stage 2 fits the longitudinal and survival submodels through the adjusted profile h-likelihood. In the random-intercept implementation, the displayed criterion is

$$\tilde{h}(\hat{\psi}) = \sum_{i=1}^N \left[h_i(\hat{u}_i; \hat{\psi}) - \frac{1}{2} \log \left\{ \frac{H_i}{2\pi} \right\} \right],$$

Here, $\hat{u}_i$ is the profiled random-intercept mode and H_i is the corresponding positive curvature. This value is stored in `x$loglik` and printed to three decimal places. It is the fitted APHL criterion, not generally an exactly marginalized log-likelihood; the Laplace adjustment is exact when the association parameter is zero.

The output contains:

Subjects The number of rows in `x$survival_data`. The fitted data structure has one survival record per subject.

Longitudinal records The number of rows in `x$longitudinal_data`.

Between-level components The retained between-subject LFPCA dimension, `x$lfpca$KB`.

Within-level components The retained within-subject LFPCA dimension, `x$lfpca$KU`.
 APHL The maximized adjusted profile h-likelihood stored in `x$loglik`.
 Optimizer convergence code The code stored in `x$convergence`. Under the optimization routines used by `lfjm`, a value of zero ordinarily indicates successful convergence. A nonzero value should be investigated together with `x$convergence_message`.

Value

The original fitted object `x`, returned invisibly. The method is called for its printed side effect.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

print(fit)

invisible_fit <- print(fit)
identical(invisible_fit, fit)

## End(Not run)
```

print.lfjm_control	<i>Print LFJM Control Parameters</i>
--------------------	--------------------------------------

Description

Prints a compact selection of the component-selection, mean-surface, quadrature, and optimization settings stored in an LFJM control object.

Usage

```
## S3 method for class lfjm_control
print(x, ...)
```

Arguments

`x` An object of class "lfjm_control", created by `lfjm_control`.
`...` Additional arguments. They are currently not used.

Details

The printed output summarizes settings governing both stages of LFJM estimation. The between- and within-level proportion-of-variance-explained thresholds are printed from `x$pve_between` and `x$pve_within`. When the corresponding component count is NULL, the method prints "selected by PVE". In that case, Stage 1 retains the smallest number of components satisfying

$$\frac{\sum_{k=1}^K \hat{\lambda}_k}{\sum_k \hat{\lambda}_k} \geq p,$$

where p is the printed PVE threshold. If a component count is supplied explicitly, that value is printed and takes precedence over the corresponding PVE rule.

Mean degree is the control value stored in `x$mean_degree` for the polynomial time trend in the functional mean surface. The printed value is the requested control, not necessarily the effective fitted degree. The mean-surface estimator also limits the degree to 3 and to the number of distinct visit times minus one.

Quadrature points gives the number of Gauss–Legendre nodes used to approximate the cumulative-hazard integral in the Stage-2 adjusted profile h-likelihood. Increasing this value can improve the numerical approximation but increases computational cost. Maximum iterations is the iteration limit supplied to each bounded optimization run.

The method prints the following fields:

Between-level PVE The target proportion of between-subject functional variance explained.

Within-level PVE The target proportion of within-subject functional variance explained.

Between components The fixed between-level component count, or an indication that PVE-based selection is used.

Within components The fixed within-level component count, or an indication that PVE-based selection is used.

Mean degree The requested polynomial-degree control for the functional mean surface.

Quadrature points The number of numerical-integration points used in Stage 2.

Maximum iterations The maximum number of iterations for each bounded optimization run.

Value

The original control object `x`, returned invisibly. The method is called for its printed side effect.

Examples

```
## Not run:
default_control <- lfjm_control()
print(default_control)

fixed_components <- lfjm_control(
  pve_between = 0.99,
  pve_within = 0.95,
  components_between = 3,
  components_within = 2,
```

```

    mean_degree = 3,
    quadrature_points = 48,
    max_iterations = 1000
  )

  print(fixed_components)

  invisible_control <- print(fixed_components)
  identical(invisible_control, fixed_components)

  fixed_components$presmooth
  fixed_components$standard_errors

  ## End(Not run)

```

print.lfjm_lfpca	<i>Print a Longitudinal Functional PCA Fit</i>
------------------	--

Description

Prints a compact overview of a longitudinal functional principal component analysis, including the sample size, grid size, retained component counts, and curve-presmoothing status.

Usage

```

## S3 method for class lfjm_lfpca
print(x, ...)

```

Arguments

x	An object of class "lfjm_lfpca", typically returned by lfjm_lfpca .
...	Additional arguments. They are currently not used.

Details

Longitudinal functional principal component analysis separates the centered functional predictor into a persistent subject-level process and a visit-specific process,

$$\tilde{X}_{ij}(s) = B_i(s) + U_{ij}(s).$$

The two processes are represented using separate truncated eigensystems. The printed between- and within-level component counts are the retained dimensions K_B and K_U , stored in `x$KB` and `x$KU`.

The method prints the following fields:

Subjects The number of unique subjects in the fitted analysis, calculated as `length(x$ids)`.

Functional grid size The number of locations in the common functional grid, calculated as `length(x$grid)`.

Between-level components The number of retained subject-level functional principal components, `x$KB`.

Within-level components The number of retained visit-level functional principal components, `x$KU`.

Curve presmoothing "yes" when `x$presmooth` is true and "no" otherwise. Presmoothing is applied to each observed functional curve before estimation of the mean surface and the two-level decomposition.

When the LFPCA was fitted with functional slopes, each between-level eigencomponent is a paired functional intercept and functional slope in a direct-sum space. In that case, the displayed value `x$KB` counts these paired components, rather than counting the intercept and slope functions separately.

Value

The original fitted object `x`, returned invisibly. The method is called for its printed side effect.

Examples

```
## Not run:
grid <- seq(0, 1, length.out = 31)
subject <- rep(1:6, each = 3)
time <- rep(c(0, 1, 2), times = 6)

subject_effect <- seq(-0.75, 0.75, length.out = 6)[subject]

functional_data <-
  matrix(
    0.25 * sin(pi * grid),
    nrow = length(subject),
    ncol = length(grid),
    byrow = TRUE
  ) +
  outer(subject_effect, sin(pi * grid)) +
  outer(0.20 * time, cos(2 * pi * grid)) +
  matrix(
    rnorm(length(subject) * length(grid), sd = 0.05),
    nrow = length(subject)
  )

lfpca_fit <- lfjm_lfpca(
  functional_data = functional_data,
  subject = subject,
  time = time,
  grid = grid,
  components_between = 2,
  components_within = 2,
  mean_degree = 3,
  presmooth = FALSE
)

print(lfpca_fit)

invisible_fit <- print(lfpca_fit)
identical(invisible_fit, lfpca_fit)

## End(Not run)
```

print.lfjm_simulation *Print Simulated LFJM Data*

Description

Prints a compact overview of a simulated longitudinal functional joint-model data set, including its simulation setting, sample dimensions, functional grid size, and observed event rate.

Usage

```
## S3 method for class lfjm_simulation
print(x, ...)
```

Arguments

x An object of class "lfjm_simulation", typically returned by `simulate_lfjm`.

... Additional arguments. They are currently not used.

Details

The simulation generator provides two functional-predictor settings. In the "longitudinal" setting, the functional predictor contains both a persistent between-subject process and visit-specific within-subject variation:

$$X_{ij}(s) = \eta(s, t_{ij}) + B_i(s) + U_{ij}(s) + e_{ij}(s).$$

In the "baseline" setting, each subject's first retained functional curve is repeated over its visits and the within-subject functional component and its coefficient are set to zero. The Setting line prints the label stored in `x$case_type`.

The method prints the following fields:

Setting The simulation case stored in `x$case_type`, currently "longitudinal" or "baseline".

Subjects The number of rows in `x$data.surv`. The simulation creates one survival record per subject.

Longitudinal records The number of rows in `x$data.long`. Scheduled visits occurring after a subject's observed follow-up time are removed, so this number can be smaller than the number of subjects multiplied by the requested number of visits.

Functional grid size The number of locations in `x$grid`.

Event rate The arithmetic mean of the binary event indicator in `x$data.surv$event`, printed to three decimal places:

$$\hat{p}_{\text{event}} = \frac{1}{N} \sum_{i=1}^N \delta_i.$$

This is the realized proportion of observed failures rather than the uncensored failure-time distribution or a hazard estimate.

Value

The original simulation object `x`, returned invisibly. The method is called for its printed side effect.

Examples

```
## Not run:
longitudinal_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

print(longitudinal_data)

baseline_data <- simulate_lfjm(
  setting = "baseline",
  number_subjects = 100,
  number_visits = 6
)

print(baseline_data)

invisible_data <- print(longitudinal_data)
identical(invisible_data, longitudinal_data)

## End(Not run)
```

print.summary.lfjm	<i>Print a Summary of an LFJM Fit</i>
--------------------	---------------------------------------

Description

Prints the model dimensions, adjusted profile h-likelihood value, optimizer convergence code, and conditional Stage-2 coefficient table stored in an "summary.lfjm" object.

Usage

```
## S3 method for class summary.lfjm
print(x, digits = 4, ...)
```

Arguments

x	An object of class "summary.lfjm", typically returned by applying summary.lfjm to a fitted "lfjm" object.
digits	A numeric value specifying the number of significant digits used to display the adjusted log-likelihood and coefficient table.
...	Additional arguments. They are currently not used.

Details

The method prints a summary of the two-stage LFPCA–APHL fit. The first part reports the numbers of subjects and longitudinal records, the retained between- and within-subject LFPCA dimensions, the maximized adjusted profile h-likelihood, and the optimizer convergence code.

At Stage 1, longitudinal functional principal component analysis decomposes the repeatedly observed functional predictor into persistent between-subject and visit-specific within-subject components. At Stage 2, the retained scores enter the longitudinal functional joint model as fixed

covariates, and the model parameters are estimated by adjusted profile h-likelihood. For a random-intercept model, the criterion has the form

$$\tilde{h}(\boldsymbol{\psi}) = \sum_{i=1}^N \left[h_i\{\hat{u}_i(\boldsymbol{\psi}); \boldsymbol{\psi}\} - \frac{1}{2} \log \left\{ \frac{H_i(\boldsymbol{\psi})}{2\pi} \right\} \right],$$

Here, $\hat{u}_i(\boldsymbol{\psi})$ is the profiled subject-specific random-intercept mode and $H_i(\boldsymbol{\psi})$ is the corresponding positive curvature. The displayed value is therefore the fitted APHL criterion and is not, in general, an exactly marginalized log-likelihood. The Laplace adjustment is exact when the current-value association parameter is zero.

The coefficient table is constructed by `summary.lfjm` and contains the following columns:

Estimate Stage-2 parameter estimates. Regression, score, and survival coefficients are displayed on their fitted scales. The residual standard deviation `sigma_e`, random-intercept standard deviation `sigma_u`, and Weibull shape parameter `Tau` are displayed on their natural positive scales.

Std.Error Conditional standard errors derived from the inverse numerical Hessian of the Stage-2 objective. For `sigma_e`, `sigma_u`, and `Tau`, the Hessian is formed on the log scale and the reported standard errors are transformed to the natural scale by the delta method.

z.value The estimate divided by its conditional standard error.

p.value The two-sided normal-approximation p-value, $2\Phi(-|z|)$.

These uncertainty summaries condition on the Stage-1 estimated mean surface, eigenfunctions, and predicted scores. They do not propagate uncertainty from the LFPCA stage. If the covariance matrix was not computed, or could not be obtained, the standard errors, z statistics, and p-values are NA. The `digits` argument changes only the printed representation; it does not modify the stored summary object.

Value

The original summary object `x`, returned invisibly. The method is called for its printed side effect.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
```

```

    grid = simulated_data$grid
  )

  fit_summary <- summary(fit)
  print(fit_summary)
  print(fit_summary, digits = 3)

  invisible_summary <- print(fit_summary)
  identical(invisible_summary, fit_summary)

  ## End(Not run)

```

residuals.lfjm

*Extract Residuals from an LFJM Fit***Description**

Extracts conditional response residuals or marginal longitudinal residuals from a fitted longitudinal functional joint model.

Usage

```

## S3 method for class lfjm
residuals(
  object,
  type = c("response", "marginal"),
  ...
)

```

Arguments

object	An object of class "lfjm", typically returned by <code>lfjm</code> .
type	A character string specifying the residual definition. Use "response" for residuals conditional on the fitted subject-specific random intercepts, or "marginal" for residuals calculated without those random intercepts. Partial matching is allowed.
...	Additional arguments. They are currently not used.

Details

Let the fitted fixed part of the longitudinal submodel be

$$\hat{\mu}_{ij}^F = \hat{\beta}_0 + \hat{\beta}_{\text{time}} t_{ij} + \hat{\beta}_Q Q_i + \hat{\xi}_i^T \hat{\theta}_B + \hat{\zeta}_{ij}^T \hat{\theta}_U,$$

The between-subject scores $\hat{\xi}_i$ and visit-specific within-subject scores $\hat{\zeta}_{ij}$ are obtained from the Stage-1 longitudinal functional principal component analysis and are treated as fixed quantities during Stage 2.

For type = "response", the method returns the conditional response residual

$$r_{ij}^C = Y_{ij} - (\hat{\mu}_{ij}^F + \hat{u}_i),$$

where $\hat{u}_i$ is the profiled subject-specific random-intercept mode from the adjusted profile h-likelihood fit. These residuals equal the observed longitudinal responses minus `fitted.lfjm` values.

For type = "marginal", the method omits the fitted random intercept and returns

$$r_{ij}^M = Y_{ij} - \hat{\mu}_{ij}^F = r_{ij}^C + \hat{u}_i.$$

The term "marginal" therefore refers specifically to exclusion of the subject-specific random intercept from the longitudinal fitted value. The functional score contributions remain in $\hat{\mu}_{ij}^F$. These are longitudinal residuals; the method does not calculate martingale, deviance, or Cox–Snell residuals for the survival submodel.

The residuals are returned in the internally ordered longitudinal-data order stored in `object$longitudinal_data`. During model fitting, this data frame and the functional-data rows are ordered jointly by subject identifier and visit time.

Value

A numeric vector with one residual for each row of `object$longitudinal_data`.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

response_residuais <- residuals(fit)
marginal_residuais <- residuals(
  fit,
  type = "marginal"
)

summary(response_residuais)
summary(marginal_residuais)

random_intercepts <- fit$uhat[
  match(
    fit$longitudinal_data$ID,
    fit$survival_data$ID
  )
]
```

```

    )
  ]

  all.equal(
    marginal_residuals,
    response_residuals + random_intercepts
  )

  ## End(Not run)

```

simulate_lfjm

Simulate Longitudinal Functional Joint-Model Data

Description

Generates data from one of two fixed longitudinal functional joint-model settings: a longitudinal setting with visit-specific functional variation or a baseline setting in which one functional curve is repeated across visits.

Usage

```

simulate_lfjm(
  setting = c("longitudinal", "baseline"),
  number_subjects = 400,
  number_visits = 8,
  grid = seq(0, 1, length.out = 101),
  visit_grid = seq(0, 5, length.out = number_visits),
  seed = NULL
)

```

Arguments

setting	A character string specifying the simulation setting. Use "longitudinal" for a repeatedly observed functional predictor with between- and within-subject variation, or "baseline" for a single subject-specific curve copied across visits. Partial matching is allowed.
number_subjects	A positive integer giving the number of subjects.
number_visits	A positive integer giving the number of planned visits per subject before event-time and censoring restrictions are applied.
grid	A finite, strictly increasing numeric vector giving the common functional observation grid. At least two grid points are required.
visit_grid	A numeric vector giving the planned visit times. Its length must equal number_visits. The default gives equally spaced visits from zero to five.
seed	An optional value supplied to set.seed before data generation. If NULL, the current random-number-generator state is used without resetting it.

Details

The function implements the two data-generating settings used to illustrate the longitudinal functional joint model. Let s denote the functional domain and let t_{ij} be the planned visit time for subject i at visit j . In the longitudinal setting, the observed functional predictor is generated as

$$X_{ij}(s) = \eta(s, t_{ij}) + B_i(s) + U_{ij}(s) + e_{ij}(s),$$

where

$$\eta(s, t) = 0.25 \sin(\pi s) + 0.10 \tilde{t} \cos(2\pi s)$$

and $\tilde{t}$ is visit time standardized over the planned visit grid. The latent components have finite Karhunen–Loeve representations

$$B_i(s) = \sum_{k=1}^3 \xi_{ik} \phi_k^B(s), \quad U_{ij}(s) = \sum_{l=1}^2 \zeta_{ijl} \phi_l^U(s).$$

The trigonometric basis functions are orthonormalized using trapezoidal integration weights on grid. The mutually independent scores satisfy $\xi_{ik} \sim N(0, \lambda_k^B)$ and $\zeta_{ijl} \sim N(0, \lambda_l^U)$. The between-level eigenvalues are (3.0, 1.5, 0.6) in both settings. In the longitudinal setting, the within-level eigenvalues are (0.5, 0.25) and the grid-error standard deviation is 0.015.

The scalar longitudinal response is generated from

$$Y_{ij} = \beta_0 + \beta_{\text{time}} t_{ij} + \beta_Q Q_i + \boldsymbol{\xi}_i^T \boldsymbol{\theta}_B + \boldsymbol{\zeta}_{ij}^T \boldsymbol{\theta}_U + u_i + \varepsilon_{ij},$$

where $Q_i \sim \text{Bernoulli}(0.5)$, $u_i \sim N(0, \sigma_u^2)$, and $\varepsilon_{ij} \sim N(0, \sigma_\varepsilon^2)$. The coefficient vectors are $\boldsymbol{\theta}_B = (0.90, -0.55, 0)^T$ and, in the longitudinal setting, $\boldsymbol{\theta}_U = (0.45, -0.30)^T$.

The survival submodel is linked to the smooth subject-specific current value

$$m_i(t) = \beta_0 + \beta_{\text{time}} t + \beta_Q Q_i + \boldsymbol{\xi}_i^T \boldsymbol{\theta}_B + u_i$$

through the Weibull proportional-hazards model

$$h_i(t) = \tau t^{\tau-1} \exp\{\gamma_0 + \gamma_Q Q_i + \alpha m_i(t)\}.$$

The within-visit functional deviation does not enter the hazard directly. Its effect is confined to the longitudinal response, whereas the functional effect induced in the log hazard is $\alpha \beta_B(s)$.

The common scalar parameter values are

$$(\beta_0, \beta_{\text{time}}, \beta_Q, \sigma_u, \sigma_\varepsilon, \tau, \gamma_0, \gamma_Q, \alpha) = (1.00, 0.28, 0.45, 0.40, 0.30, 1.20, -4.20, -0.45, 0.90).$$

Event times are obtained by numerical inversion of the cumulative hazard. The censoring time is the minimum of an exponential random variable with rate 0.05 and administrative censoring at time 8. Only planned visits satisfying $t_{ij} \leq T_i$, where T_i is the observed event or censoring time, are retained. Consequently, the returned number of longitudinal records can be smaller than `number_subjects * number_visits`.

For `setting = "baseline"`, the within-level coefficient is zero and the first retained functional curve for each subject is copied exactly to all of that subject's retained visits. The core generator uses negligible within-level eigenvalues (10^{-8} , 10^{-8}) and grid-error standard deviation 10^{-4} before the first curve is copied. The returned true within-subject curves and within-level coefficient function are set exactly to zero. This setting represents the degenerate case in which the functional predictor is time invariant.

Value

An object of class `c("lfjm_simulation", "list")` with the following components:

`data.long` A data frame containing the retained longitudinal records, ordered by subject and visit time. It contains `ID`, `obstime`, `row_id`, `Q1`, `time`, `event`, `Y`, and `true_mu_BU`. The observed functional predictor is stored in columns `func.X.1`, `func.X.2`, and so on. The variables `time` and `event` repeat the subject-level survival outcome on each retained visit. The variable `true_mu_BU` is the true conditional longitudinal mean, including the random intercept.

`data.surv` A data frame containing one row per subject and the variables `ID`, observed follow-up time, event indicator `event`, scalar covariate `Q1`, and true random intercept `true_u`.

`grid`, `sgrid` Identical numeric vectors containing the functional grid.

`true_B_visit` A numeric matrix containing the true between-subject functional component for each retained row of `data.long`. Its columns correspond to `grid`.

`true_U_visit` A numeric matrix containing the true visit-specific within-subject functional component for each retained row of `data.long`. It is an all-zero matrix in the baseline setting.

`truth` A list containing the generating values `beta0`, `beta_time`, `beta_Q`, `sigma_u`, `sigma_eps`, `Tau`, `gamma0`, `gamma_Q`, and `alpha`; score coefficients `theta_B` and `theta_U`; grid-evaluated coefficient functions `beta_B_true` and `beta_U_true`; the induced survival functional effect `surv_B_current_true`; basis matrices `phi0` and `phiU`; and the subject-level between scores `xi`.

`case_type` A character string equal to `"longitudinal"` or `"baseline"`.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6,
  seed = 2026
)

print(simulated_data)
names(simulated_data$truth)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

print(fit)
summary(fit)
```



```

baseline_data <- simulate_lfjm(
  setting = "baseline",
  number_subjects = 100,
  number_visits = 6,
  seed = 2026
)

print(baseline_data)

## End(Not run)

```

summary.lfjm

*Summarize a Longitudinal Functional Joint Model Fit***Description**

Constructs a summary of a fitted longitudinal functional joint model, including conditional Stage-2 coefficient estimates, standard errors, Wald statistics, fit information, and retained functional component counts.

Usage

```

## S3 method for class lfjm
summary(object, ...)

```

Arguments

object An object of class "lfjm", typically returned by `lfjm`.

... Additional arguments. They are currently not used.

Details

The longitudinal functional joint model is fitted in two stages. Stage 1 estimates the mean surface, between- and within-subject eigensystems, and functional principal component scores. Stage 2 treats these estimated quantities as fixed covariates and estimates the joint-model parameters by adjusted profile h-likelihood.

For retained between-subject scores $\hat{\xi}_i$ and within-subject scores $\hat{\zeta}_{ij}$, the working longitudinal sub-model is

$$Y_{ij} = \beta_0 + \beta_{\text{time}} t_{ij} + \beta_Q Q_i + \hat{\xi}_i^T \theta_B + \hat{\zeta}_{ij}^T \theta_U + u_i + \varepsilon_{ij}.$$

The survival hazard is

$$h_i(t) = \tau t^{\tau-1} \exp\{\gamma_0 + \gamma_Q Q_i + \alpha m_i(t)\},$$

where the shared current value is

$$m_i(t) = \beta_0 + \beta_{\text{time}} t + \beta_Q Q_i + \hat{\xi}_i^T \theta_B + u_i.$$

The rows of the coefficient table are ordered as follows:

1. the longitudinal intercept, visit-time coefficient, and scalar longitudinal-covariate coefficient, named "(Intercept)", "obstime", and "Q1";
2. the retained between-level score coefficients, named "Bscore1", "Bscore2", and so forth;

3. the retained within-level score coefficients, named "Uscore1", "Uscore2", and so forth; and
4. "sigma_e", "sigma_u", "gamma0", "gamma_Q", "alpha", and "Tau".

Thus, the table reports the finite-dimensional score coefficients θ_B and θ_U . It does not contain the reconstructed grid-level coefficient functions $\hat{\beta}_B(s)$, $\hat{\beta}_U(s)$, or the induced survival functional effect $\hat{\alpha}\hat{\beta}_B(s)$. These can be extracted with `coef.lfjm`.

When `object$covariance_matrix` is available, standard errors are calculated from the diagonal of the inverse numerical Hessian of the minimized negative Stage-2 criterion. The optimization parameterization uses $\log(\sigma_e)$, $\log(\sigma_u)$, and $\log(\tau)$. The corresponding natural-scale standard errors are therefore obtained by the first-order delta method:

$$\text{se}(\hat{\sigma}_e) = \hat{\sigma}_e \text{se}\{\log(\hat{\sigma}_e)\}, \quad \text{se}(\hat{\sigma}_u) = \hat{\sigma}_u \text{se}\{\log(\hat{\sigma}_u)\},$$

and

$$\text{se}(\hat{\tau}) = \hat{\tau} \text{se}\{\log(\hat{\tau})\}.$$

The remaining standard errors are taken directly from the optimization-scale covariance matrix. For every displayed parameter estimate $\hat{\vartheta}$, the method forms

$$z = \frac{\hat{\vartheta}}{\text{se}(\hat{\vartheta})}, \quad p = 2\Phi(-|z|).$$

If the covariance matrix was not requested or could not be computed, all standard errors, z statistics, and p-values are returned as NA.

The reported `logLik` is the maximized adjusted profile h-likelihood,

$$\tilde{h}(\hat{\psi}) = \sum_{i=1}^N \left[h_i(\hat{u}_i; \hat{\psi}) - \frac{1}{2} \log \left\{ \frac{H_i}{2\pi} \right\} \right],$$

Here, $\hat{u}_i$ is the profiled random-intercept mode and H_i is its positive curvature. The value is a Laplace-adjusted profile criterion rather than, in general, an exactly marginalized log-likelihood; the adjustment is exact when the association parameter is zero.

Value

An object of class "summary.lfjm", represented by a list with the following components:

`call` The matched call used to fit the model.

`coefficients` A data frame with rows corresponding to the Stage-2 model parameters and columns Estimate, Std.Error, z.value, and p.value.

`logLik` The maximized adjusted profile h-likelihood value.

`objective` The minimized negative adjusted profile h-likelihood value returned by the optimizer.

`convergence` The optimizer convergence code. Under the optimization routines used by `lfjm`, zero ordinarily indicates successful convergence.

`convergence_message` The optional optimizer convergence message.

`number_subjects` The number of subject-level survival records.

`number_longitudinal_records` The number of retained longitudinal observations.

`components_between`, `components_within` The numbers of retained between- and within-subject LFPCA components.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6,
  seed = 2026
)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid
)

fit_summary <- summary(fit)
print(fit_summary)

fit_summary$coefficients
fit_summary$convergence
fit_summary$components_between
fit_summary$components_within

## End(Not run)
```

vcov.lfjm

*Extract the Covariance Matrix from an LFJM Fit***Description**

Extracts the conditional Stage-2 inverse-Hessian covariance matrix from a fitted longitudinal functional joint model.

Usage

```
## S3 method for class lfjm
vcov(object, ...)
```

Arguments

object An object of class "lfjm", typically returned by [lfjm](#).

... Additional arguments. They are currently not used.

Details

Let $\boldsymbol{\vartheta}$ denote the Stage-2 optimization parameter vector and let $Q(\boldsymbol{\vartheta}) = -\tilde{h}(\boldsymbol{\vartheta})$ be the minimized negative adjusted profile h-likelihood. When standard errors are requested, the fitting procedure evaluates the numerical Hessian at the optimizer:

$$\hat{\mathcal{I}} = \nabla_{\boldsymbol{\vartheta}}^2 Q(\boldsymbol{\vartheta})|_{\boldsymbol{\vartheta}=\hat{\boldsymbol{\vartheta}}}.$$

The returned covariance matrix is

$$\hat{\mathcal{V}} = \hat{\mathcal{I}}^{-1}.$$

The ordinary matrix inverse is used when possible. If it fails, the fitting procedure attempts to use a generalized inverse. Consequently, an ill-conditioned fit can yield a covariance matrix that should be interpreted cautiously even when extraction succeeds.

The matrix is returned on the *optimization scale*. Its row and column names follow the parameter order

$$\boldsymbol{\vartheta} = \left(\boldsymbol{\beta}^T, \log \sigma_e, \log \sigma_u, \gamma_0, \gamma_Q, \alpha, \log \tau \right)^T,$$

Here, $\boldsymbol{\beta}$ contains, in order, the longitudinal intercept, visit-time effect, scalar-covariate effect, retained between-level score coefficients, and retained within-level score coefficients. Their matrix names are "(Intercept)", "obstime", "Q1", "Bscore1", and so forth, followed by "Uscore1" and so forth. The remaining names are "log_sigma_e", "log_sigma_u", "gamma0", "gamma_Q", "alpha", and "log_Tau".

In particular, the entries associated with the residual standard deviation, random-intercept standard deviation, and Weibull shape parameter do not directly give natural-scale variances. Let

$$g(\boldsymbol{\vartheta}) = \left(\boldsymbol{\beta}^T, \sigma_e, \sigma_u, \gamma_0, \gamma_Q, \alpha, \tau \right)^T$$

be the natural-scale transformation, with $\sigma_e = \exp(\log \sigma_e)$, $\sigma_u = \exp(\log \sigma_u)$, and $\tau = \exp(\log \tau)$. If $\mathbf{J}$ is the Jacobian of g , the first-order delta-method covariance matrix is

$$\hat{\mathcal{V}}_{\text{natural}} = \mathbf{J} \hat{\mathcal{V}} \mathbf{J}^T.$$

The relevant diagonal elements of $\mathbf{J}$ are $\hat{\sigma}_e$, $\hat{\sigma}_u$, and $\hat{\tau}$; all other diagonal elements are one. [summary.lfjm](#) applies these univariate delta-method transformations when reporting the standard errors of the three positive parameters.

This covariance matrix is based on the curvature of the adjusted profile h-likelihood. It is not a sandwich covariance estimator and does not propagate uncertainty from the Stage-1 estimation of the mean surface, eigensystems, or functional scores. The matrix therefore describes conditional Stage-2 uncertainty under the fitted APHL criterion.

Value

A named square numeric matrix with one row and column for every Stage-2 optimization parameter. The matrix is expressed on the optimization scale.

Examples

```
## Not run:
simulated_data <- simulate_lfjm(
  setting = "longitudinal",
  number_subjects = 100,
  number_visits = 6,
  seed = 2026
```

```

)

functional_columns <- paste0(
  "func.X.",
  seq_along(simulated_data$grid)
)

functional_data <- as.matrix(
  simulated_data$data.long[, functional_columns, drop = FALSE]
)

fit <- lfjm(
  longitudinal_data = simulated_data$data.long,
  survival_data = simulated_data$data.surv,
  functional_data = functional_data,
  grid = simulated_data$grid,
  control = lfjm_control(
    standard_errors = TRUE
  )
)

covariance_optimizer <- vcov(fit)
sqrt(diag(covariance_optimizer))

# Transform the complete covariance matrix to the natural parameter scale.
jacobian <- diag(nrow(covariance_optimizer))
dimnames(jacobian) <- dimnames(covariance_optimizer)

jacobian["log_sigma_e", "log_sigma_e"] <- fit$sigma_e
jacobian["log_sigma_u", "log_sigma_u"] <- fit$sigma_u
jacobian["log_Tau", "log_Tau"] <- fit$Tau

covariance_natural <-
  jacobian
  covariance_optimizer
  t(jacobian)

natural_names <- rownames(covariance_optimizer)
natural_names[natural_names == "log_sigma_e"] <- "sigma_e"
natural_names[natural_names == "log_sigma_u"] <- "sigma_u"
natural_names[natural_names == "log_Tau"] <- "Tau"

dimnames(covariance_natural) <- list(
  natural_names,
  natural_names
)

sqrt(diag(covariance_natural))

## End(Not run)

```

Index

`coef.lfjm`, [2](#), [42](#)

`fitted.lfjm`, [4](#), [37](#)

`lfjm`, [2](#), [4](#), [6](#), [16](#), [18](#), [22](#), [28](#), [29](#), [36](#), [41–43](#)

`lfjm_control`, [7](#), [9](#), [30](#)

`lfjm_lfpca`, [7](#), [12](#), [20](#), [21](#), [25](#), [31](#)

`logLik.lfjm`, [16](#)

`matplot`, [18](#), [20](#)

`plot`, [18](#), [20](#)

`plot.lfjm`, [17](#)

`plot.lfjm_lfpca`, [19](#)

`predict.lfjm`, [22](#)

`predict.lfjm_lfpca`, [25](#)

`print.lfjm`, [28](#)

`print.lfjm_control`, [29](#)

`print.lfjm_lfpca`, [31](#)

`print.lfjm_simulation`, [33](#)

`print.summary.lfjm`, [34](#)

`residuals.lfjm`, [36](#)

`set.seed`, [38](#)

`simulate_lfjm`, [33](#), [38](#)

`smooth.spline`, [10](#), [13](#)

`summary.lfjm`, [34](#), [35](#), [41](#), [44](#)

`vcov.lfjm`, [43](#)